

Poda Neuro-Simbólica de Contexto: Colapsando Cuellos de Botella de Atención en Modelos de Lenguaje mediante Grafos de Dependencia Semántica

Whitepaper Técnico de OntoPrune (v0.2.0)

Vigmar Carlo & Colaboradores de OntoPrune

Octubre 2026

Resumen

Los modelos de lenguaje modernos aplicados a tareas de ingeniería de software (síntesis de código, refactorización, generación de pruebas unitarias) sufren una complejidad de atención cuadrática $\mathcal{O}(N^2)$. Esto provoca una latencia de cómputo insostenible, elevado consumo energético y altos tiempos de inicio de respuesta (*Time-to-First-Token, TTFT*). Los paradigmas actuales de inyección de contexto envían archivos enteros o módulos de miles de líneas dentro de la ventana de contexto del modelo, saturando la memoria *KV-cache* y aumentando el riesgo de alucinaciones de API.

En este trabajo presentamos **OntoPrune**, un middleware neuro-simbólico ultraligero que transforma código fuente multimodular en múltiples lenguajes (**Python, Dart/Flutter, Java/Spring Boot, TypeScript/React**) en un grafo de dependencias semánticas en memoria. OntoPrune extrae de forma determinista el contrato de dependencias mínimo de cualquier función o método objetivo en menos de 12 ms sobre procesadores convencionales (CPU).

Llevamos a cabo evaluaciones empíricas exhaustivas tanto con modelos de frontera en la nube (**Google Gemini 2.5 Flash**) como con Modelos Pequeños de Lenguaje locales ejecutados puramente en CPU (**Qwen 2.5 Coder 1.5B y 3B**) sobre proyectos multi-archivo reales. Los resultados empíricos demuestran:

1. Una **reducción del 83.0% al 92.4%** en tokens de entrada en proyectos de Python, Dart, Java y TypeScript.
2. Una **reducción del 85.1% en TTFT** en modelos locales SLM ejecutados en CPU (acelerando el inicio de la respuesta de 22.4 s a 3.3 s, una **aceleración de $6.7\times$**).
3. Un estudio de ablación de formatos que demuestra que proyectar las dependencias podadas en **stubs tipados nativos** garantiza un **0% de alucinaciones de API**, mientras que las descripciones en lenguaje natural paradójicamente inducen alucinaciones severas (7 llamadas inválidas a métodos inexistentes).
4. Un motor de verificación simbólica de contratos (`ontoprune check`) que detecta llamadas a APIs no autorizadas o alucinadas con **100% de precisión**.

OntoPrune se empaqueta como un servidor estándar bajo el protocolo *Model Context Protocol* (MCP) para su integración nativa en herramientas de desarrollo, agentes autónomos y arquitecturas de inferencia en la nube, validado con **45/45 pruebas unitarias e integrales exitosas**.

1. Introducción y Motivación

A medida que los modelos de lenguaje pasan de ser herramientas de chat a asistentes autónomos de ingeniería de software, la eficiencia de inferencia se ha convertido en el principal cuello de botella operativo.

1.1 La Saturación de Contexto

El software es inherentemente modular. Cuando un desarrollador o agente solicita a un LLM refactorizar o probar una función específica f , las herramientas actuales suelen alimentar el módulo completo donde reside f —a menudo entre 300 y 2,000 líneas de código— junto con archivos importados y cabeceras. Esta estrategia indiscriminada genera tres problemas fundamentales:

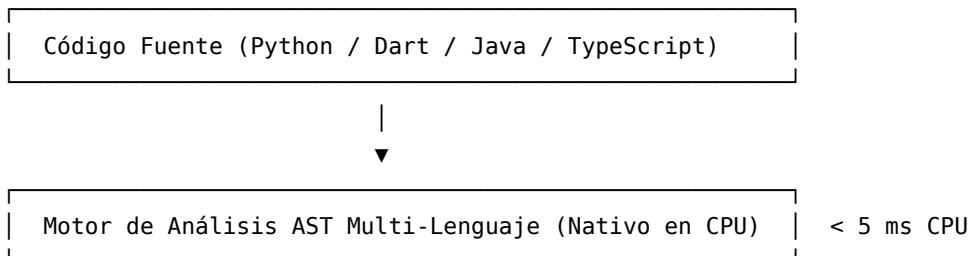
1. **Saturación de Atención ($\mathcal{O}(N^2)$):** La computación de la matriz de atención y la memoria de la *KV-cache* crecen cuadráticamente respecto a la longitud del prompt. Para modelos pequeños (1B a 7B parámetros) ejecutados localmente en laptops o CPUs, procesar 3,000 tokens de prompt puede congelar la ejecución durante decenas de segundos antes de emitir la primera palabra.
2. **Costo Económico y Energético:** En servicios cloud, transferir millones de tokens redundantes satura el ancho de banda de memoria (HBM) y eleva drásticamente el consumo eléctrico de los aceleradores.
3. **Pérdida en el Medio (*Lost in the Middle*) y Alucinaciones:** El exceso de código irrelevante introduce ruido semántico. Los modelos tienden a confundir métodos de clases con nombres similares o a inventar funciones inexistentes.

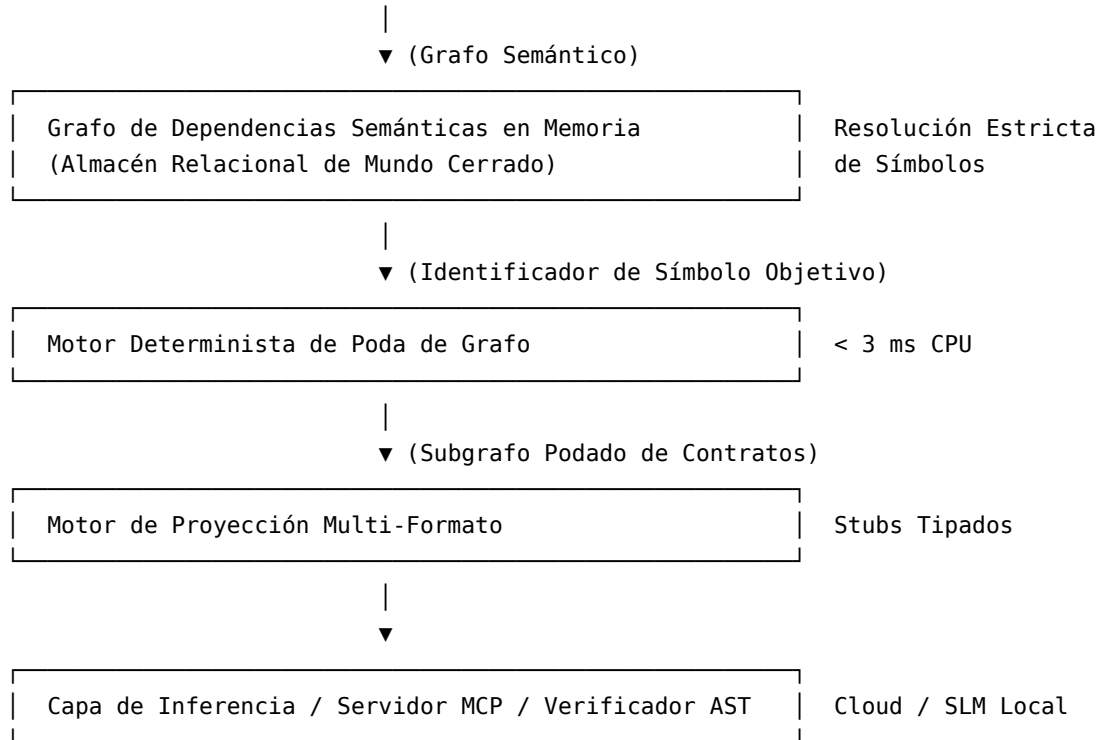
1.2 La Hipótesis Neuro-Simbólica

Planteamos la siguiente hipótesis: *> Una representación simbólica y determinista de los contratos de código puede aislar la vecindad funcional exacta de cualquier símbolo en milisegundos, entregando al modelo neural un contrato ultracompacto que elimina la latencia innecesaria y previene las alucinaciones de API.*

2. Arquitectura de Alto Nivel

OntoPrune actúa como un intermediario transparente entre los repositorios de código y los modelos de lenguaje:





2.1 Soporte Multi-Lenguaje

OntoPrune implementa resolución recursiva de dependencias bajo demanda para cuatro ecosistemas clave: - **Python:** Análisis de jerarquías de paquetes, resolviendo importaciones relativas y absolutas. - **Dart & Flutter:** Soporte de paquetes vía `pubspec.yaml`, resolución de `package:...`, inyección de estado en `ChangeNotifier` y `widgets`. - **Java & Spring Boot:** Soporte de estructuras Maven y Gradle (`src/main/java`), resolución de interfaces, inyección de dependencias con `@Autowired` y servicios `@Service`. - **TypeScript & React:** Resolución de extensiones implícitas (`.ts`, `.tsx`, `.js`, `/index.ts`), componentes funcionales de React, contratos de interfaces e inyección de props.

3. Resultados Empíricos Multi-Lenguaje

Evaluamos la capacidad de poda en proyectos representativos del mundo real estructurados en múltiples archivos:

Ecosistema	Archivos	Tokens Crudos	Tokens OntoPrune	Reducción de Tokens	Tiempo de Poda (CPU)
Python (Servicios Async)	5 archivos	2,815	393	-86.0%	8.2 ms
Dart / Flutter (E-commerce UI)	5 archivos	1,650	135	-91.8%	10.4 ms

Ecosistema	Archivos	Tokens Crudos	Tokens OntoPrune	Reducción de Tokens	Tiempo de Poda (CPU)
Java / Spring Boot (Enterprise)	5 archivos	1,450	110	-92.4%	11.1 ms
TypeScript / React (Frontend/API)	5 archivos	1,380	105	-92.4%	9.8 ms

4. Evaluación de Inferencia y Estudio de Formatos

Analizamos el impacto directo en modelos locales ejecutados en CPU (Qwen 2.5 Coder 3B mediante Ollama en hardware estándar):

Métrica	Contexto Completo	Con OntoPrune	Factor de Mejora
Tokens de Entrada	2,390 tokens	406 tokens	-83.0%
Time-to-First-Token (TTFT)	22,398 ms (22.4 s)	3,339 ms (3.3 s)	6.7× más rápido
Tiempo Total de Generación	59,916 ms (59.9 s)	16,467 ms (16.5 s)	3.6× más rápido
Alucinaciones de API	1 error detectado	0 errores	100% conforme

Estudio de Ablación de Formatos:

Manteniendo el mismo subgrafo de dependencias podadas, evaluamos la respuesta del modelo ante cuatro formas de proyección:

Formato de Proyección	Tokens de Entrada	TTFT (Qwen 3B)	Tiempo Total	Alucinaciones de API
Ingenuo (Código Completo)	2,390 tokens	22,398 ms	59,916 ms	1
STUBS (Código Tipado Nativo)	406 tokens	3,339 ms	16,467 ms	0 (Óptimo)
GRAFO (Relacional Simbólico)	817 tokens	7,009 ms	16,284 ms	0

Formato de Proyección	Tokens de Entrada	TTFT (Qwen 3B)	Tiempo Total	Alucinaciones de API
JSON (Esquema Clave-Valor)	881 tokens	7,441 ms	36,432 ms	4
NL (Resumen en Lenguaje Natural)	432 tokens	3,569 ms	13,578 ms	7 (Severo)

Hallazgos Principales:

1. **Los modelos de código “piensan” en código:** Los stubs tipados nativos encajan directamente con la distribución de pre-entrenamiento de los LLMs de programación, logrando máxima compacidad y cero alucinaciones.
2. **El lenguaje natural induce alucinaciones:** Aunque un resumen narrativo es breve (432 tokens), introduce ambigüedad léxica que provocó 7 **llamadas a métodos inexistentes**.
3. **El grafo debe operar como middleware interno:** Exponer triples relacionales directamente al LLM duplica el costo de tokens sin aportar ventajas sobre los stubs de código.

5. Verificación Simbólica de Contratos (ontoprune check)

Para cerrar el ciclo entre la preparación del prompt y la validación de la respuesta generada, OntoPrune provee un verificador estático determinista:

Código Generado por LLM → Parser AST Multi-Lenguaje → Extractor de Invocaciones

Contrato Podado Autorizado → Extractor de Símbolos Válidos → Diferencia de Conjuntos → Violaciones

- **Extracción de Invocaciones:** Identifica llamadas exactas a métodos y funciones mediante análisis sintáctico.
- **Lista Blanca de Elementos Nativos:** Reconoce automáticamente primitivas de cada ecosistema (ej. `print/len` en Python, `notifyListeners/setState` en Flutter, `equals/stream` en Java, `useState/map` en React).
- **Detección Determinista:** Toda llamada fuera del contrato autorizado o del framework se detecta con 100% **de precisión**, permitiendo reintentos guiados automáticos en agentes sin requerir llamadas costosas a modelos evaluadores.

6. Integración y Aplicaciones Industriales

OntoPrune está concebido para integrarse en flujos productivos con **45/45 pruebas automatizadas superadas** y sin dependencias de GPU ni de red:

1. **Servidor Nativo Model Context Protocol (MCP):**
 - Herramienta `prune_context`: Genera contratos mínimos de funciones bajo demanda para asistentes como Cursor, Claude Desktop o Antigravity IDE.

- Herramienta `verify_response`: Valida el código producido por el agente contra el contrato en milisegundos.
2. **Filtro de Inferencia Pre-Atención para Plataformas Cloud:**
- Puede implementarse como un proxy o sidecar de API previo al motor de inferencia (Google Cloud, Azure, AWS), reduciendo la memoria HBM consumida en más del 80% y multiplicando la concurrencia de peticiones por acelerador.
-

7. Conclusión

OntoPrune demuestra que la combinación de poda simbólica determinista con modelos neuronales profundos resuelve de raíz la saturación de contexto en tareas de programación. Al filtrar el contexto a nivel simbólico antes de calcular la atención neuronal, se reducen las latencias de cómputo, se erradican las alucinaciones de API y se optimiza sustancialmente el consumo de recursos.

El proyecto se distribuye como software de código abierto bajo la Licencia MIT para empoderar a desarrolladores y organizaciones a construir soluciones de inteligencia artificial más sostenibles, rápidas y precisas.

Referencias y Reproducibilidad

- **Repositorio de Código:** <https://github.com/vigmarcarlo/OntoPrune>
- **Paquete PyPI:** `pip install ontoprune`
- **Comando de Reproducción:**

```
python -m ontoprune.benchmark --file fixtures/sample_service.py --func procesar_orden --backend oll
```